

Claude Code Quick Start

Your First 60 Minutes: From Setup to Professional Workflow

Brandon Behring | February 2026

For data scientists and developers new to Claude Code

By the end of this hour, you will have:

1. Claude Code installed and running in your project
2. Your first AI-assisted test (a real test in your test suite)
3. Your first AI-assisted commit (a real commit in your history)
4. A `CLAUDE.md` that encodes your project's standards
5. A clean session close with context for next time

Four terms you'll encounter (defined again in the margins on first use):

Session	Your conversation with Claude Code, persisted across terminal restarts.
Tool use	When Claude runs a command, reads a file, or edits code on your behalf.
CLAUDE.md	A file where you give Claude persistent instructions about your project.
Permission	Whether Claude asks before acting or acts autonomously. Default mode always asks first.

Tip

Already comfortable with Claude Code? Skip to *Your First CLAUDE.md* (Section 3.1) — the essential commands table is on the last page.

1 Safety

1.1 Install and Launch (Minutes 0–5)

Install Claude Code following the instructions at claude.ai/download, then launch it in your project:

```
cd your-project
claude
```

```
Welcome to Claude Code!
/help for commands, /quit to exit
```

>

You have a prompt. Claude is ready.

If Something Goes Wrong

- **Auth error** — Run `claude login` and follow the prompts.
- **Wrong directory** — Claude works best in a project root with source files.
- **No project handy** — Try it on any git repo, even a small one.

1.2 Your First Question (Minutes 5–10)

Ask Claude to explain your codebase:

Vocab

Session — your conversation with Claude, persisted across terminal restarts. Resume anytime with `claude`
--continue.

You type: "What does this project do? Focus on the main entry points and data flow."

This project is a demand forecasting pipeline with three main components:

- Data ingestion: `src/data/load.py`
- Feature engineering: `src/features/build.py`
- Model training: `src/models/train.py`

The entry point is `main.py`, which orchestrates...

Now ask something specific:

You type: "@src/features/build.py explain this function's edge cases — what happens with empty input?"

Claude returns a real explanation of *your* code. It didn't guess — it used tools to examine your source files. Notice: the second prompt was more specific and got a better answer. That's the specification skill.

1.3 Safety Net and First Edit (Minutes 10–15)

Before we ask Claude to change anything — your safety nets:

Your Safety Net

Esc	Stop Claude mid-action. Context preserved, nothing lost.
Esc+Esc	Open rewind menu. Restore conversation, code, or both.
Approval	In Default mode, Claude always asks before editing. You approve or reject every change.

Now, your first edit:

You type: "Add a one-line docstring to `build_features()` describing what it does and what it returns."

Claude reads the function, writes the docstring, and shows you a diff. A permission dialog appears. Read the diff — does it match the function's actual behavior? If yes, approve.

Tip

Notice how Claude broke this into steps: read the function, understand it, write the docstring, ask for approval. That's an *agent loop* — you'll see the full cycle diagrammed shortly. It's the same pattern you'd use when designing agents for your team.

Precision Matters

Vague: "Add some documentation"

→ Claude guesses which functions, what style, how verbose

Precise: "Add a one-line docstring to `build_features()` describing what it does and what it returns"

→ One function, one change, verifiable result

Tip

The @ prefix loads file content before Claude responds — more precise than describing what to read.

Cross-Ref

For the full vocabulary of precision prompting, see [Prompting That Works](#).

Tip

Nothing is permanent until you approve it. If anything goes wrong, Esc + Esc rewinds to any previous state.

Official

Five permission modes. Shift+Tab cycles Default, Accept Edits, Plan, dontAsk and bypassPermissions via settings/CLI.

<https://code.claude.com/docs/en/permissions>

Vocab

Tool use — each file read, edit, or command Claude runs is a tool call. You just saw three: read the file, write the docstring, show the diff.

The Shift

You just described what you wanted and verified the result — without writing a line of code yourself. That's the new loop: *specify* what you want, *observe* what Claude produces, *refine* if needed. The implementation is Claude's job. Knowing what "correct" looks like is yours.

Now let's make "correct" concrete — with tests.

2 Understanding

2.1 Explore, Then Test (Minutes 15–25)

The core loop: explore → implement → verify.

Step 1 – Explore:

You type: "Explain what `build_features()` does, including edge cases. What happens with empty input? What about missing keys?"

Claude explains the function's behavior, including cases you may not have considered.

Step 2 – Write a test:

You type: "Write a test for `build_features()` that covers:
 - Normal input with expected output
 - Empty input
 - Missing required keys
 Put it in `tests/test_features.py`"

Claude writes the test file. Permission dialog appears. Approve it.

Step 3 – Run the test:

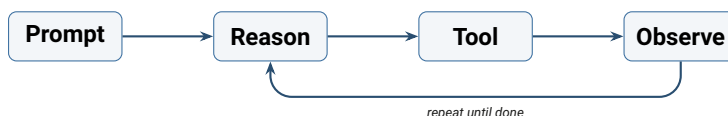
You type: "Run the test."

```
tests/test_features.py::test_normal_input PASSED
tests/test_features.py::test_empty_input PASSED
tests/test_features.py::test_missing_keys PASSED
3 passed in 0.42s
```

Your code now has a regression test it didn't have five minutes ago. If tests *fail* — even better. Claude just found a bug. The test caught a real issue. This is exactly why we test.

Why Testing Is the Core Skill

AI-generated code has a specific failure mode: it *looks correct*. Clean syntax, reasonable names, plausible logic. The bugs are subtle, not obvious. Your ability to define what "correct" means — through tests — is now more valuable than your ability to write the implementation.

**Official**

"Give Claude a way to verify its own work" is Anthropic's stated single highest-leverage practice.

<https://code.claude.com/docs/en/best-practices>

Tip

Tests constrain implementation. When Claude writes code to pass *your* tests, the "looks correct but is subtly wrong" failure mode is caught automatically. Full TDD workflow: [The Edit-Test-Commit Loop](#).

You just experienced this loop: you prompted (“write a test”), Claude reasoned about the function, used tools (Read, Write, Bash/pytest), and you observed the result.

Tip

You defined what “correct” means for this function. In agent design, these become guardrails — the verification criteria that keep autonomous systems producing trustworthy results.

2.2 Review, Regression, and Commit (Minutes 25–35)

Step 1 – Review the diff:

You type: "Show me all the changes we've made this session."

Reading Diffs – What to Check

- Does the docstring match the function’s *actual* behavior?
- Does the test import the right module?
- Are there any changes you didn’t expect?
- Does the test actually assert correctness, not just “doesn’t crash”?

Step 2 – Regression check:

You type: "Run the full test suite to make sure our changes didn't break anything."

This takes 30 seconds and catches the most expensive bugs — the ones where a fix in one place breaks something elsewhere.

Step 3 – Commit:

You type: "Commit these changes with message:
'Add docstring and tests for build_features'"

Claude stages the files, writes the message, and commits. It used multiple tools in sequence — diff, stage, commit — without you specifying each step. You described the *outcome*; Claude figured out the *procedure*. That’s the specification skill: describe what, not how.

From Builder to Architect

You didn’t write the test or the docstring — but you defined what they should do, verified the result, and caught a real issue. Your value shifted from *typing code* to *defining correctness*. That’s not a demotion — it’s an upgrade. Architects who can verify are more valuable than typists who can implement.

Now let’s make these practices permanent.

Vocab

Regression testing — verifying that new code doesn’t break existing behavior. The practice that turns a collection of scripts into reliable software.

Cross-Ref

For context management during long sessions, see [Context as Currency](#).

3 Ownership

3.1 Your First CLAUDE.md (Minutes 35–45)

You type: /init

Claude generates a starter CLAUDE.md from your project structure. Review it — it's a starting point, not finished. Let's make it yours.

Add three things:

1. Build commands — tell Claude how to verify:

```
## Build & Verify
- `pytest tests/` - run test suite
- `ruff check src/` - lint (if applicable)
```

2. Testing standard — make it non-negotiable:

```
## Standards
- Write tests before implementation
- Run tests after every code change
```

3. Documentation standard — one line is enough:

```
- Every public function must have a docstring
```

Here's what a mature CLAUDE.md looks like after a few sessions of refinement:

```
# Demand Forecasting

## Build & Verify
- `pytest tests/` - run test suite
- `mypy src/` - type checking
- `ruff check src/` - lint

## Architecture
- src/features/ - feature engineering pipelines
- src/models/ - model training and evaluation
- src/data/ - data loading and validation
- tests/ - mirrors src/ structure

## Standards
- Type hints on all function signatures
- Run tests after every code change
- Never commit without passing lint + tests
```

Warning

Do NOT write: *"This is an ML project. We use scikit-learn. Tests are important."*

DO write: **commands**, **locations**, and **rules** — not descriptions. Claude needs to know how to *work on* your project, not *about* it.

Tip

CLAUDE.md is your first agent configuration file. Every agent you build — for your team, for a pipeline, for production — will need something like it: persistent instructions that shape behavior without requiring human presence.

Your standards are now infrastructure, not memory. Every future session starts from this baseline.

Official

CLAUDE.md loads automatically every session. It is the single most important configuration file.

<https://code.claude.com/docs/en/best-practices>

Tip

Keep CLAUDE.md under 300 lines. Move module-specific rules to `.claude/rules/` with path filters. Full architecture: [CLAUDE.md Architecture](#).

3.2 Close Cleanly (Minutes 45–55)

Step 1 – Name the session:

You type: `/rename first-session-demand-forecast`

Step 2 – Check context:

You type: `/context`

Step 3 – Clear for next task:

You type: `/clear`

Fresh context for the next task. The session is saved — resume anytime.

Three Rules That Prevent 90% of Frustration

1. **One task per session.** `/clear` between unrelated work.
2. **Two-failure rule.** After two failed corrections → `/clear` + better prompt. Don't push through noise.
3. **Tests, not trust.** If Claude's code looks right but has no test, it's not done.

Tip

Treat sessions like branches — one logical workstream per session. Resume later with `claude --continue` or `claude --resume`.

Warning

When context crosses ~60%, quality starts to drop. `/compact` or `/clear` proactively. Full context toolkit: [Context as Currency](#).

Infrastructure Not Memory

You encoded your standards into `CLAUDE.md` so they persist without you remembering them. You named your session so you can find it later. You cleared context so the next task starts clean. This is the collaboration skill: designing systems that work reliably — whether the collaborator is Claude, an agent you build, or a future teammate.

Cross-Ref

For the full anti-patterns catalog (eight traps and their escapes), see [Anti-Patterns and Recovery](#).

3.3 What You Built (Minutes 55–60)

In one hour, you:

- Explored your codebase with Claude (specification)
- Wrote a real test and caught real behavior (verification)
- Ran regression tests to ensure nothing broke (engineering discipline)
- Committed with a meaningful message (workflow)
- Created `CLAUDE.md` with project standards (collaboration)
- Closed cleanly with a named session (professional habit)

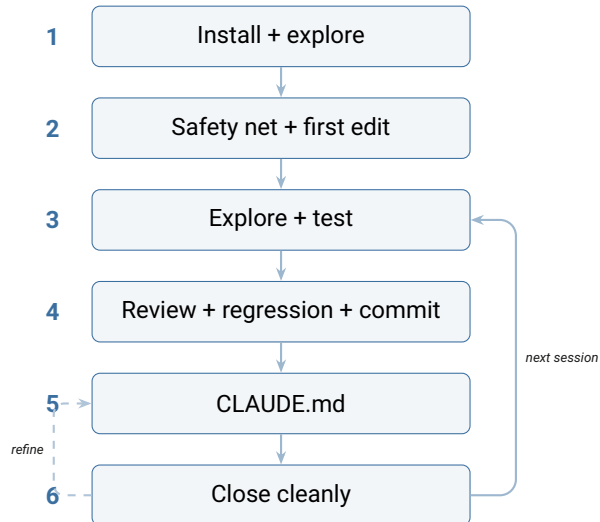
Everything you practiced today — specification, verification, collaboration design — is the foundation of effective AI-assisted development. The skills transfer directly to building agents, designing pipelines, and working with AI tools beyond Claude Code. The handbook's 16 chapters deepen every practice you touched in this hour.

Where to go next:

Testing Full TDD workflow, 6-layer validation, property-based testing. See [The Edit-Test-Commit Loop](#).

Context Session strategies, status line, CURRENT_WORK.md, auto-memory. See [Context as Currency](#).

CLAUDE.md Rules files, settings layers, hooks for enforcement. See [CLAUDE.md Architecture](#).



Essential Commands

Tip

Bookmark this page. The commands table is your quick reference for every session.

Command / Shortcut	What It Does
/init	Generate starter CLAUDE.md from project
/clear	Clear context, start fresh
/compact	Summarize, preserve key decisions
/context	Visualize current context usage
/rewind	Restore conversation, code, or both to a checkpoint
Esc	Stop Claude mid-action (context preserved)
Esc + Esc	Open rewind menu (restore or summarize from checkpoint)
Shift+Tab	Cycle modes (Default → Accept Edits → Plan) ¹
Alt+T	Toggle extended thinking
/debug	Inspect model, session, and configuration state
/status	Show version, account, and plan info
--continue	Resume most recent session
--resume	Pick from previous sessions

Extracted from "Best Practices for Using Claude" (v2.3+).

Full handbook: 16 chapters covering foundations through enterprise deployment.

Licensed under CC BY 4.0 | Content verified against Anthropic docs, February 2026

¹dontAsk and bypassPermissions available via settings.